



SENAI - SERVIÇO NACIONAL DE APRENDIZAGEM INDÚSTRIAL
APRENDIZAGEM INDUSTRIAL EM PROGRAMADOR DE SISTEMAS DA INFORMAÇÃO
DOCENTE: LUCAS DE MATTOS

RELATÓRIO DE CRIAÇÃO E MANIPULAÇÃO DE BANCO DE DADOS

JHIONATAN R. DE FREITAS

SALTO VELOSO - SC
08.10.2026

SUMÁRIO

1. INTRODUÇÃO.....	3
2. DESENVOLVIMENTO.....	3
2. TIPO DE BANCO DE DADOS UTILIZADOS.....	3
2.1 TABELAS E O QUE SÃO.....	4
2.2 VIEWS.....	5
2.3 TRIGGERS.....	6
2.4 PROCEDURES	7

1. INTRODUÇÃO

Este relatório consiste em apresentar as estruturas que foram abordadas e praticadas dentro do curso de **Criação e Manipulação de Banco de Dados**, desde o tipo do banco de dados (DB), tabelas, procedures, views e triggers, explicando cada uma delas e como foram trabalhadas dentro do curso através do **Projeto CMDDB**.

2. TIPO DE BANCO DE DADOS UTILIZADO

Durante a aprendizagem foi utilizado o banco de dados relacional, no qual consiste em uma estrutura organizada os dados em tabelas com chaves primárias e estrangeiras. Diferente do banco de dados não relacional que armazena os dados de forma livre em formato de **JSON**, não contendo tabelas ou chaves primárias e estrangeiras. Usamos o Sistema Gerenciador de Banco de Dados Relacional o **My SQL** de código aberto que utiliza a linguagem padrão SQL (**Structured Query Language**) para criar e manipular os dados.

Exemplo de uma tabela

Colunas: Adicionar Remove Mover para cima Mover para baixo											
#	Nome	Tipo de dados	Tamanho/Itens	Unsign...	Permitir...	Zero fill	Padrão	Comentário	Colação	Expressão	Virtualidade
	1 id	INT		☐	☐	☐	AUTO_INCREME...				
2	nome	VARCHAR	100	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		
3	cnj_cpf	VARCHAR	18	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		
4	rua	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_general_ci		
5	numero	VARCHAR	50	☐	☒	☐	NULL		utf8mb4_general_ci		
6	bairro	VARCHAR	100	☐	☒	☐	NULL		utf8mb4_general_ci		
7	cep	VARCHAR	10	☐	☒	☐	NULL		utf8mb4_general_ci		
8	referencia	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_general_ci		
9	telefone	VARCHAR	100	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		

O banco de dados foi desenvolvido de forma relacional para garantir integridade, organização e consistência dos dados, se alinhando adequadamente no projeto que é um sistema **E-Commerce**. Se fosse um sistema de finanças ou gestão de usuários, qualquer sistema que envolve informações transacionais, isto é, dados sendo gerados em tempo real.

2.1 TABELAS E O QUE SÃO

O **Banco de Dados Relacional**, é formado por tabelas que são estruturas que contém linhas e colunas que organizam e armazenam os dados. Cada coluna é um campo, deve ressaltar que existem diferentes tipos de dados e cada campo irá conter apenas um tipo de dado. Tipos servem para armazenar informações, como por exemplo **Varchar** que armazena apenas dados de texto de tamanho de até 255 caracteres.

Tabela com tipo Varchar

#	Nome	Tipo de dados	Tamanho/Itens	Unsign...	Permitir...	Zero fill	Padrão	Comentário	Colação	Expressão	Virtualidade
1	id	INT		☐	☐	☐	AUTO_INCREME...				
2	nome	VARCHAR	255	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		
3	cnpj_cpf	VARCHAR	18	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		
4	rua	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_general_ci		
5	numero	VARCHAR	50	☐	☒	☐	NULL		utf8mb4_general_ci		
6	bairro	VARCHAR	100	☐	☒	☐	NULL		utf8mb4_general_ci		
7	cep	VARCHAR	10	☐	☒	☐	NULL		utf8mb4_general_ci		
8	telefone	VARCHAR	100	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		

As tabelas contém chaves **primárias** e **secundárias**, essencial para identificar e conectar as tabelas. A chave primária (**Primary Key**) é o identificador único de cada linha. A chave estrangeira (**Foreign Key**) serve para relacionar cada uma delas. As tabelas contém apenas uma chave primária, porém para chave estrangeira pode haver muito mais do que apenas uma.

Tabela contendo chave primária e chaves estrangeiras

#	Nome
1	id
2	id_cliente
3	id_produto

2.2 VIEWS

Durante a aprendizagem além criar e popular as tabelas, também foi abordado as **Views** e as **Triggers**. São duas coisas diferentes mas que se relacionam. O Banco de Dados contém diversas informações que serão manipuladas ou consultadas, porém há um determinado ponto as consultas se tornam grandes demais, e para maior rapidez e eficiência é criado as **Views**.

Uma **View** (visão), é uma tabela virtual no qual o conteúdo está definido por uma consulta SQL que já foi criada, assim consultas que demorariam muito tempo para serem realizadas, com uma view elas podem serem feitas em um tempo incomparavelmente menor.

Exemplo de uma view

```
1 CREATE VIEW resumo_clientes AS
2 SELECT
3     c.id,
4     c.nome,
5     COUNT(p.id) AS total_pedidos,
6     SUM(p.valor) AS valor_total
7 FROM clientes c
8 LEFT JOIN pedidos p ON p.cliente_id = c.id
9 GROUP BY c.id, c.nome;
```

2.3 TRIGGERS

Além das **Views** temos as **Triggers** que servem para trazer eficiência e integridade dos dados. Elas servem para atualizar as informações de uma tabela, ao invés de ter que haver uma pessoa para fazer essa atualização, o próprio banco de dados faz esse processo com segurança sem que esses dados apresentem problemas.

Porém o uso excessivo pode deixar o banco de dados lento, pois muitas linhas estarão sendo processadas de uma só vez, e pode dar Bugs ou erros de processamento

Exemplo de uma Trigger

```
1 BEGIN
2     INSERT INTO tmp_historico_produtos(
3         produto_id,
4         nome_antigo,
5         preco_antigo,
6         estoque_antigo,
7         usuario_alteracao
8     ) VALUES (
9         OLD.id,
10        OLD.nome,
11        OLD.valor,
12        OLD.estoque,
13        USER()
14    );
15 END
```

2.4 PROCEDURES

Além das **Views**, **Triggers**, Criação e manipulação do banco de dados, também estudamos os **Procedures**, que não é muito diferente das **Views**. Elas são uma espécie de blocos que são utilizadas para guardar linhas de código que em algum momento irá precisar executar novamente.

Além de trazer maior eficiência pois o DB não irá precisar reescrever muitas linhas de código e também ajuda no processamento, pois o código está **Pré-Compilado**.

Exemplo de uma Procedure

```
1 DELIMITER $$
2
3 CREATE PROCEDURE inserir_aluno(
4     IN p_nome VARCHAR(50),
5     IN p_idade INT,
6     IN p_endereco VARCHAR(100)
7 )
8 BEGIN
9     INSERT INTO alunos (nome, idade, endereco)
10    VALUES (p_nome, p_idade, p_endereco);
11 END $$
12
13 DELIMITER ;
```